

INSTITUTE OF ENGINEERING & MANAGEMENT

NEWTOWN, KOLKATA

Department of Electronics & Communication
Engineering

Module 2: Project Report

Analysis of Real-Time Object Detection Using YOLO

Submitted by: PARTHIBA SARKAR

Enrollment Number: 12023002002106

Registration Number: 304202300900493

Class Roll Number: B-33

1. Introduction

Analysis of Real-Time Object Detection Using YOLO

Object detection is an important part of computer vision and autonomous vehicle systems. It allows a system to identify and locate objects such as people, cars, buses, motorcycles, and trucks in images and videos. In this project, YOLO (You Only Look Once) is used to perform object detection on images and a video. The detected objects are displayed using bounding boxes and labels.

2. Objective

- To implement object detection using YOLO.
- To detect multiple objects in images and videos.
- To analyze YOLO's detection performance and processing speed.
- To demonstrate its use for real-time autonomous vehicle perception.

3. Technologies Used

- Python 3.11
- YOLO (Ultralytics)
- OpenCV
- VS Code
- YOLO26n model
- CPU-based processing

4. Methodology

The YOLO model was loaded using the Ultralytics Python library. Test images and a video were provided as input. YOLO processed the input and identified objects by drawing bounding boxes and assigning object labels. For video processing, detection was performed frame by frame and the processed video was saved as output.

5. Implementation

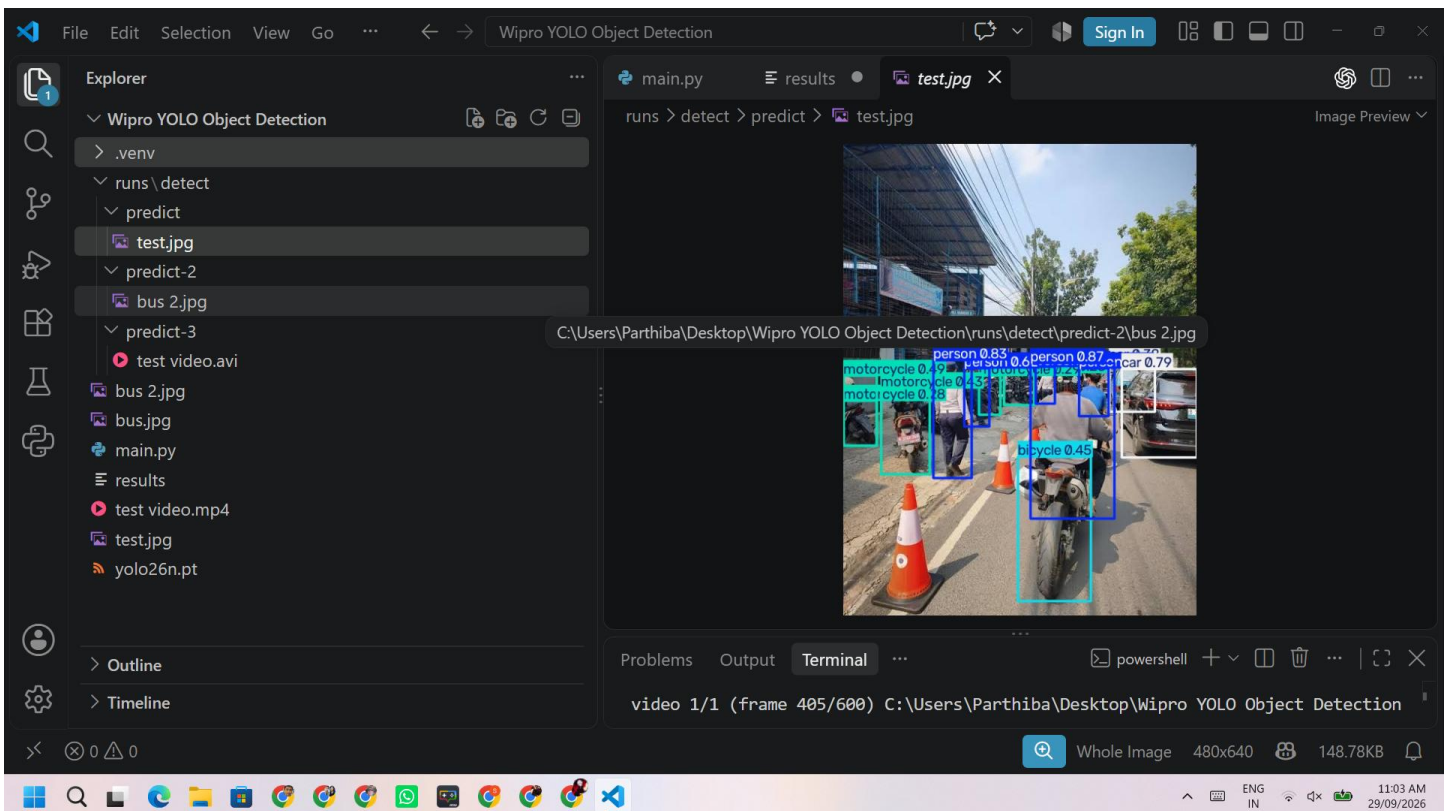
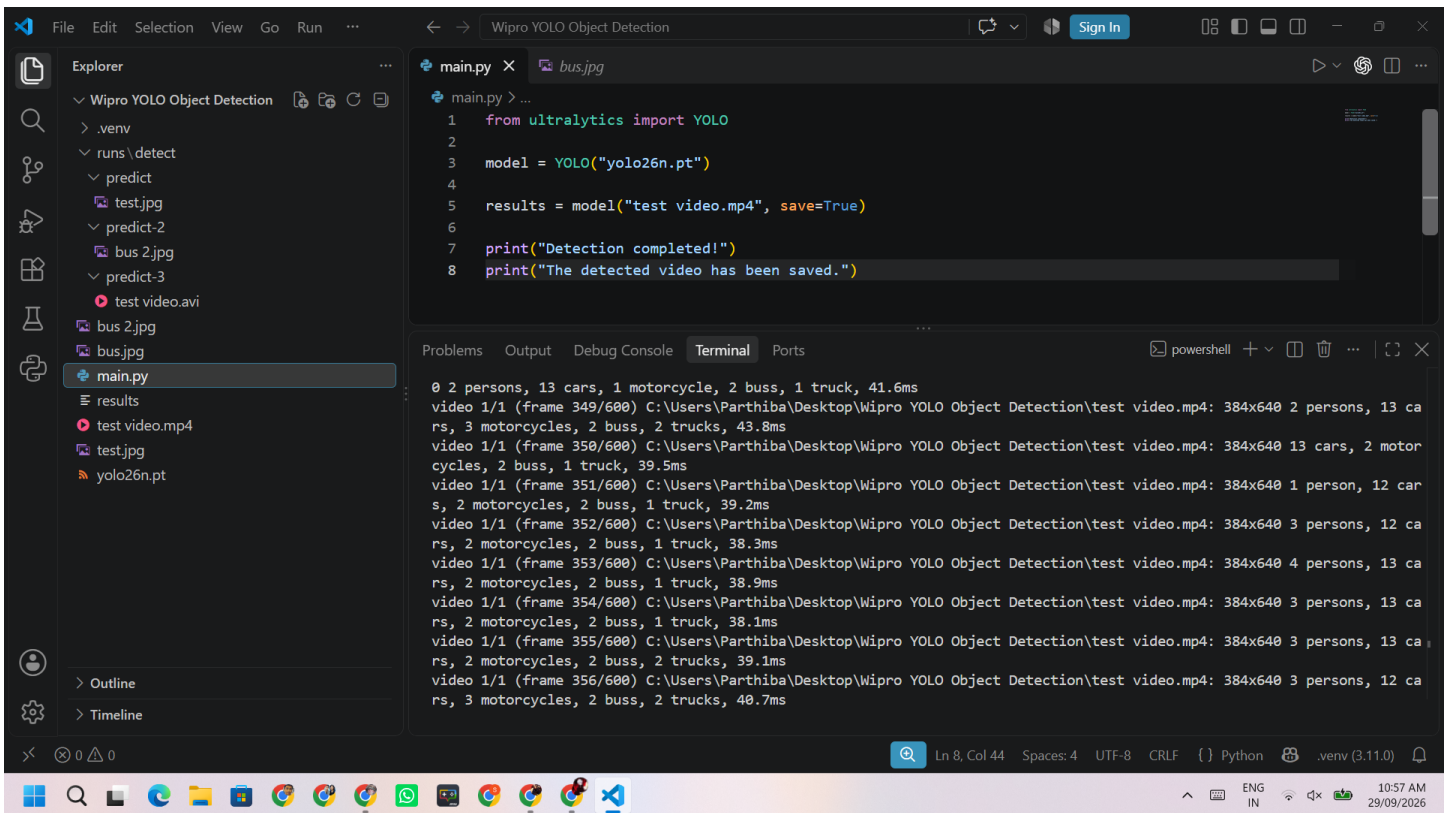
The project was implemented using Python and the Ultralytics YOLO library. A pretrained `yolo26n.pt` model was loaded and tested on two images and one video.

The following inputs were used:

- `test.jpg`
- `bus 2.jpg`

- test video.mp4

YOLO detected objects in the inputs and generated bounding boxes with their corresponding labels. The save=True option was used to save the processed images and video in the runs/detect output folder.



6. Results and Analysis

The YOLO model was successfully tested on both images and video.

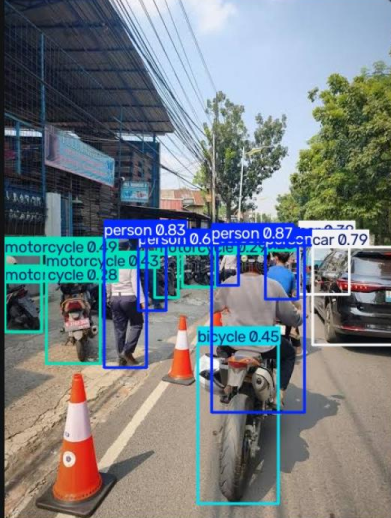
- **test.jpg:** Detected 5 persons, 1 bicycle, 2 cars, and 5 motorcycles with an inference time of approximately **72.9 ms**.
- **bus 2.jpg:** Detected 1 person and 2 buses with an inference time of approximately **68.3 ms**.
- **test video.mp4:** The model processed **600 frames** at a resolution of **384 × 640**. Objects such as persons, cars, motorcycles, buses, and trucks were detected across different frames. The processed video was successfully saved as output.

The results show that YOLO can detect multiple object classes and provide their locations using bounding boxes.

Wipro YOLO Object Detection

main.py test.jpg

runs > detect > predict > test.jpg



video 1/1 (frame 352/600) C:\Users\Parthiba\Desktop\Wipro YOLO Object Detection\test video.mp4: 384x640 3 persons, 12 ca

Whole Image 480x640 148.78KB

Wipro YOLO Object Detection

main.py bus 2.jpg

runs > detect > predict-2 > bus 2.jpg



video 1/1 (frame 352/600) C:\Users\Parthiba\Desktop\Wipro YOLO Object Detection\test video.mp4: 384x640 3 persons, 12 ca

Whole Image 630x420 105.45KB

Wipro YOLO Object Detection

main.py bus.jpg

bus.jpg

Image Preview

video 1/1 (frame 352/600) C:\Users\Parthiba\Desktop\Wipro YOLO Object Detection\test video.mp4: 384x640 3 persons, 12 ca

Whole Image 810x1080 134.20KB

10:57 AM 29/09/2026

Wipro YOLO Object Detection

main.py test video.mp4

test video.mp4

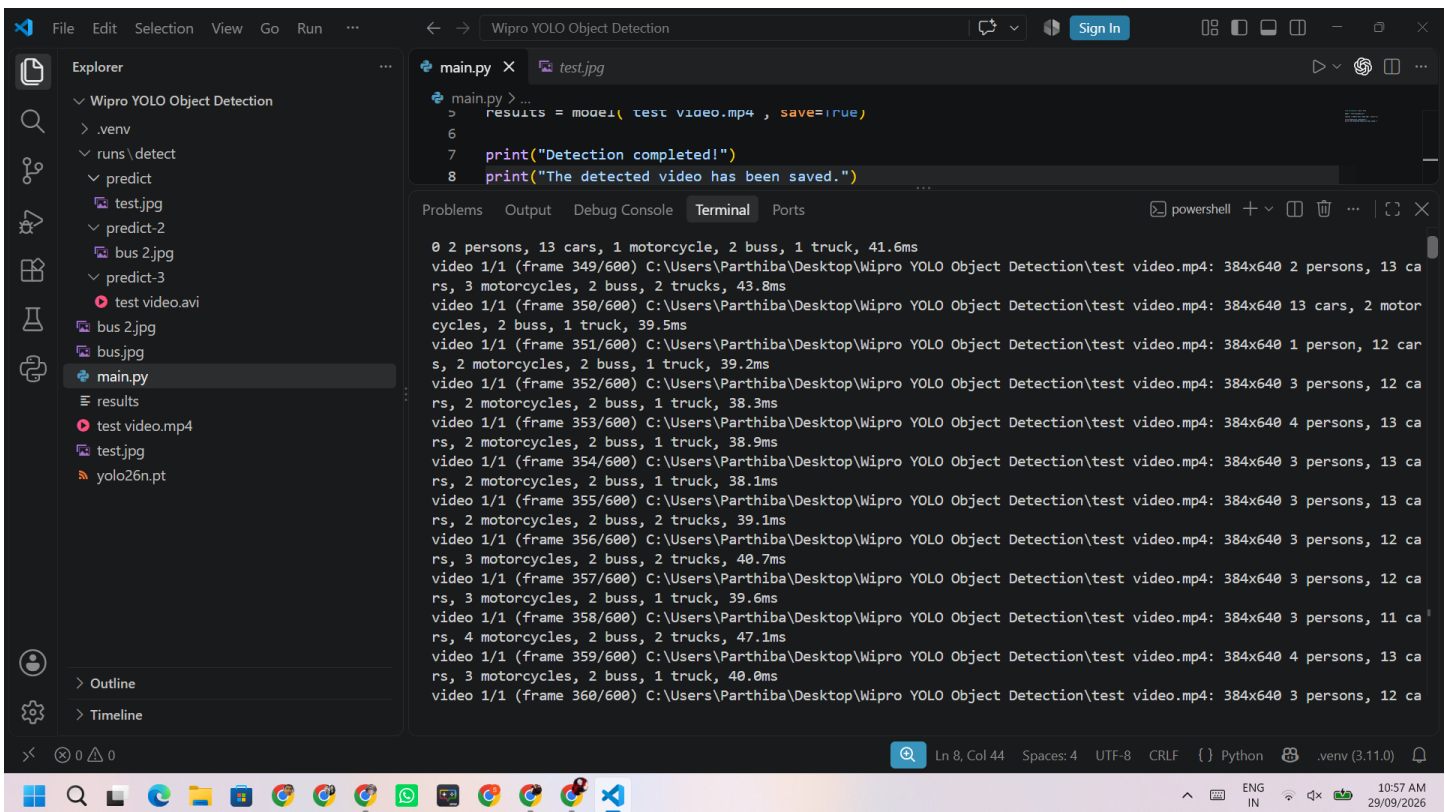
Video Preview

0:00 / 0:12

video 1/1 (frame 352/600) C:\Users\Parthiba\Desktop\Wipro YOLO Object Detection\test video.mp4: 384x640 3 persons, 12 ca

4.56MB

10:58 AM 29/09/2026



7. Conclusion

The project successfully implemented **YOLO-based object detection** for images and video. The system was able to identify different objects and generate annotated outputs with bounding boxes and labels. This demonstrates how YOLO can be used as a basic **perception component in autonomous vehicle systems**.

8. Future Scope

- Use GPU processing for faster detection.
- Evaluate accuracy using precision, recall, and mAP.
- Add real-time camera input.
- Implement object tracking.
- Integrate the system with autonomous-driving simulators such as CARLA or ROS.

9. References

1. Ultralytics, “YOLO Documentation”
<https://docs.ultralytics.com/>
2. Ultralytics, “Model Prediction with YOLO”
<https://docs.ultralytics.com/modes/predict/>

3. Ultralytics, “Object Detection”
<https://docs.ultralytics.com/tasks/detect/>
4. Ultralytics, “Python Usage”
<https://docs.ultralytics.com/usage/python/>

GitHub Repository:

<https://github.com/parthibasarkar/Wipro-YOLO-Object-Detection>

In GitHub Repository I have uploaded the assignment